Customer Engagement & Service Analytics Platform

Capstone Project Report

WIPRO NGA - .Net React (MS Dynamics) – FY26 – C3

Submitted By:

Mamidanna Jaya Surya Karthik

Table of Contents:

1. Introduction 1
2. Problem Definition 1
3. System Design 2
4. Component Breakdowns & API Design 12
5. Database Design & Storage Optimization 17
6. Reporting & Analytics 21
7. Conclusion 23

1. Introduction:

In today's fast-paced digital environment, organizations must efficiently manage customer interactions and resolve service requests in a timely manner. Customer support systems play a crucial role in ensuring customer satisfaction by tracking issues, assigning responsibilities, and monitoring resolution progress.

This project focuses on designing and implementing a Customer Engagement and Service Analytics System using SQL Server and Power BI. The system is built on a normalized database schema consisting of Customers, Tickets, TicketCategories, and Agents to ensure data integrity, consistency, and efficient querying. It enables structured storage of customer issues, assignment of tickets to support agents, and categorization based on problem types.

In addition to database design, the project integrates Power BI to provide interactive dashboards and visual insights. These analytics help in monitoring key performance indicators such as ticket volume, resolution time, agent performance, and customer activity. The overall goal of this project is to demonstrate how data-driven solutions can enhance customer service efficiency and support better decision-making in organizations.

2. Problem Definition:

Customer Support teams require an efficient system to manage support requests. Traditional methods such as email and spreadsheets make it difficult to track ticket status, resolution time, and support workload.

Objectives:

- Build a web-based ticket management system.
- Allow customers to create support tickets.
- Allow agents to resolve and manage tickets.
- Monitor ticket statistics using dashboards.
- Provide operational reporting using SSRS

3. System Design:

3.1. Overview of Architecture:

The Customer Support Ticket Management System is designed using a three-tier architecture, which separates the application into three main layers: the presentation layer (frontend), the application layer (backend), and the data layer (database).

This architecture improves scalability, maintainability, and performance by ensuring that each layer handles a specific responsibility.

The frontend is developed using, React, the backend is implemented using ASP.NET Core Web API, and the database is managed using Microsoft SQL Server. Data visualization and reporting are achieved using Power BI and SQL Server Reporting Services (SSRS).

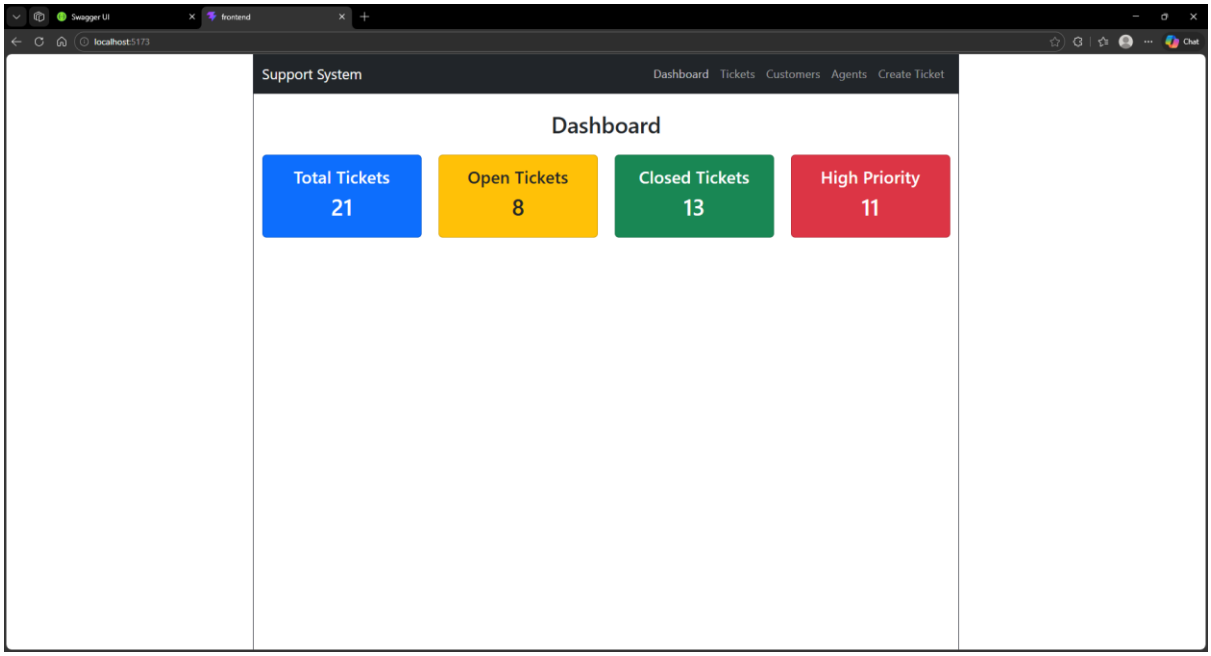
3.2. System Architecture:

1. Frontend Layer (Presentation Layer):

The frontend layer is developed using React and is responsible for providing the user interface of the application.
It allows users to:

- View all tickets.
- Create new support tickets.
- Resolve or delete tickets.
- View customer and agent data.

The frontend communicates with the backend using HTTP requests (GET, POST, PUT, DELETE) through REST APIs.



This is the Dashboard page of the frontend

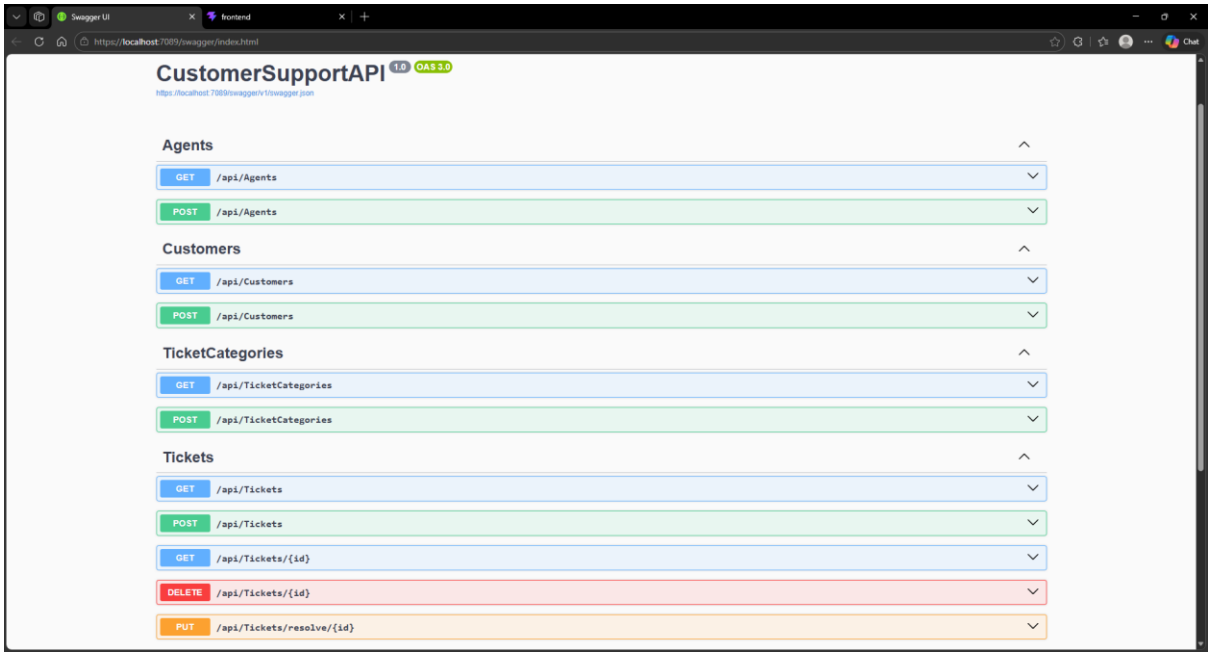
2. Backend Layer (Application Layer):

The backend is developed using ASP.NET Core Web API and acts as the bridge between the frontend and the database.
It follows a layered architecture:

- Controller Layer – Handles incoming HTTP requests
- Service Layer – Contains business Logic
- Repository Layer – Handles database operations

The backend exposes RESTful APIs such as:

- GET /api/Tickets
- POST /api/Tickets
- PUT /api/Tickets/resolve/{id}
- DELETE /api/Tickets/{id}



Swagger UI backend

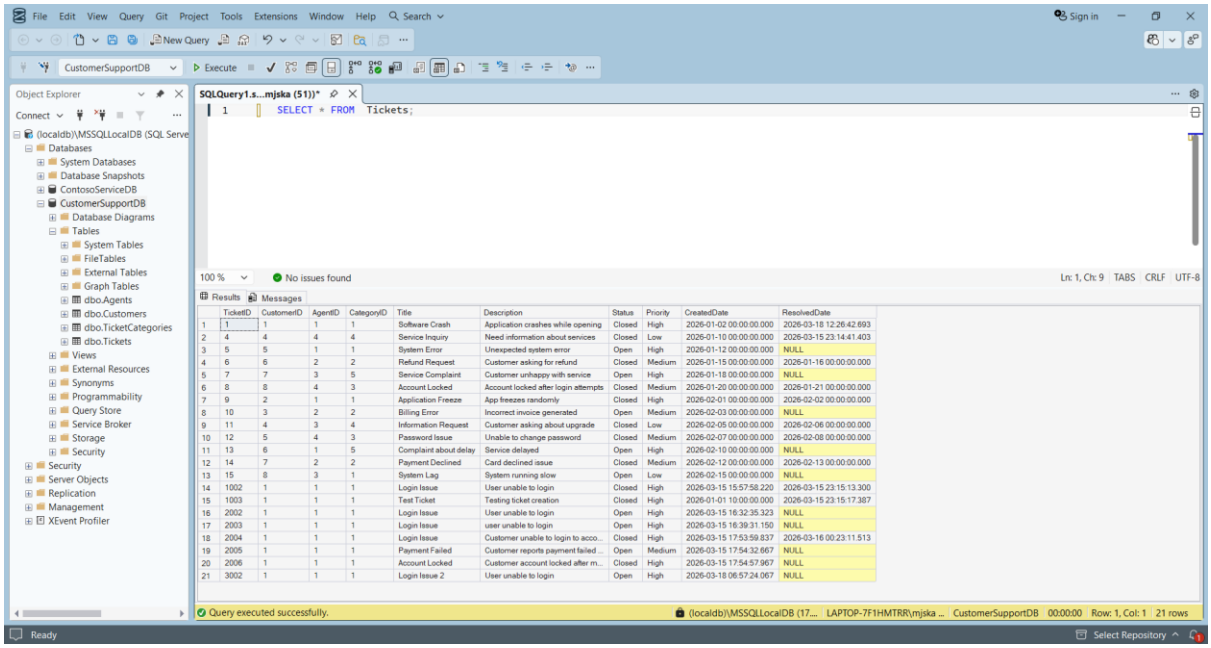
3. Database Layer (Data Layer):

The database layer is implemented using Microsoft SQL Server and stores all application data.

The main tables include:

- Customers
- Agents
- Tickets
- Ticket Category

The backend interacts with the database using Entity Framework Core, which simplifies data access and object mapping.



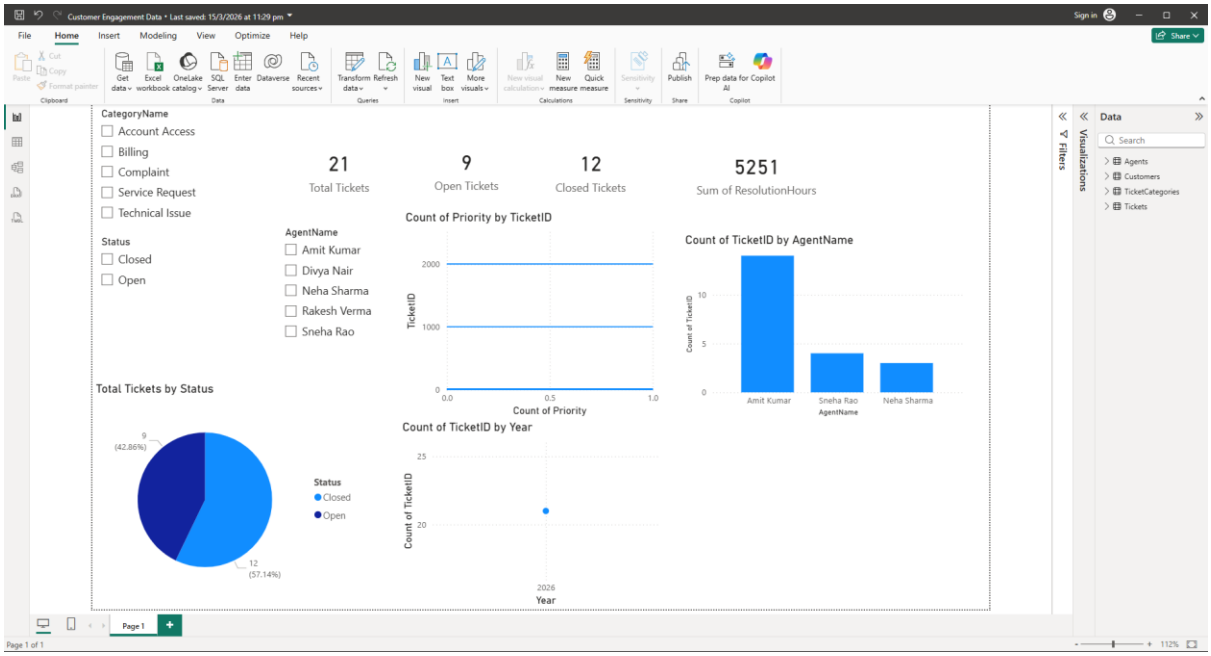
SSMS

4. Reporting & Analytics Layer:

The system includes reporting and analytics using Power BI and SQL Server Reporting Services (SSRS).

Power BI is used to create interactive dashboards that display:

- Ticket distribution by status
- Ticket trends over time
- Priority analysis



Power BI Dashboard

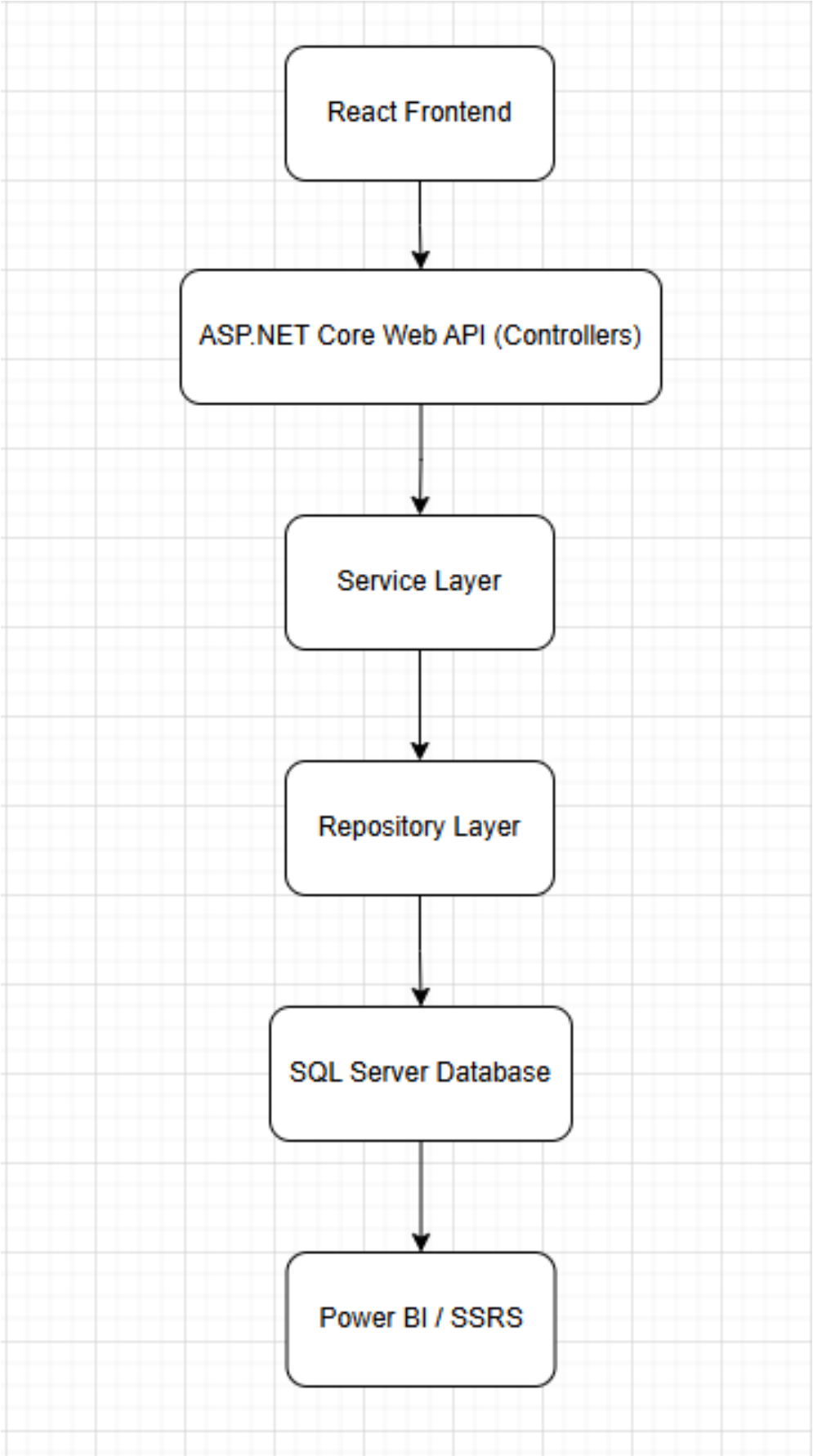
SSRS is used to generate operational reports such as:

- Ticket volume report
- Resolution time report
- SLA Compliance report

These tools help managers monitor system performance and make data-driven decisions.

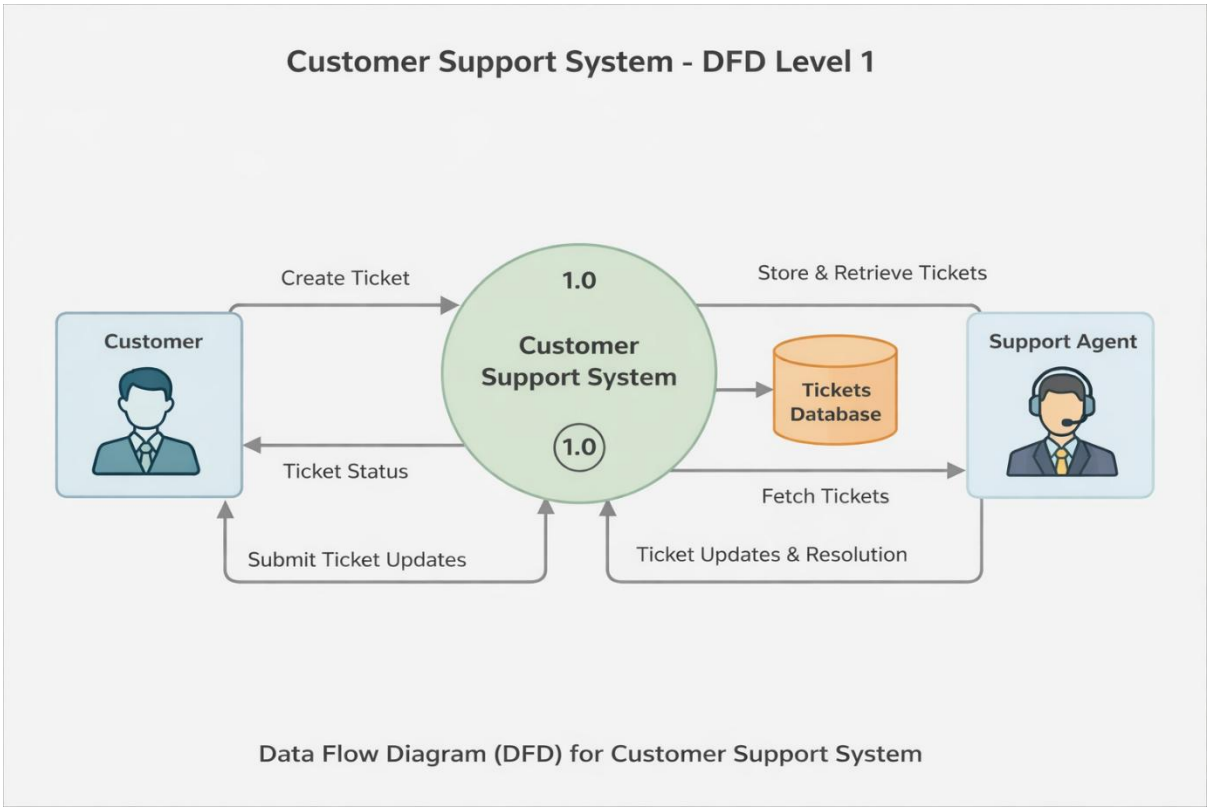
SSRS

Architecture Flow Diagram:



The above diagram represents the flow of data in the system. The React frontend sends requests to the ASP.NET Core API. The API processes the request through service and repository layers, interacts with the SQL Server database, and returns the response to the frontend. Data is further used for analytics in Power BI and reporting in SSRS.

3.3. Data Flow Diagram:



The Data Flow Diagram (DFD) illustrates how data moves within the Customer Support System. Customers create service tickets, which are processed and stored in the database. Support agents retrieve, update, and resolve tickets. The system ensures proper tracking and provides data for reporting and analytics through Power BI dashboards.

Data Flow Explanation:

When a user creates a ticket, the frontend sends a POST request to the backend API. The controller receives the request and passes it to the service layer, where business logic is applied. The repository layer then stores the ticket in the SQL Server database.

Similarly, when fetching tickets, the frontend sends a GET request, and the backend retrieves data from the database and returns it as JSON.

For resolving or deleting tickets, PUT and DELETE requests are used respectively.

4. Component Breakdown & API Design:

4.1. Overview:

This section explains the internal structure of the Customer Support Ticket Management System. The application is divided into multiple components on both the frontend and backend to ensure modularity, reusability, and maintainability.

The frontend consists of reusable React components, while the backend is structured using controllers, services, and repositories following a layered architecture.

4.2. Frontend Component Breakdown:

The frontend of the application is developed using React and is divided into multiple components, each responsible for a specific functionality.

Main components include:

•App Component
Acts as the root component and manages routing and layout.

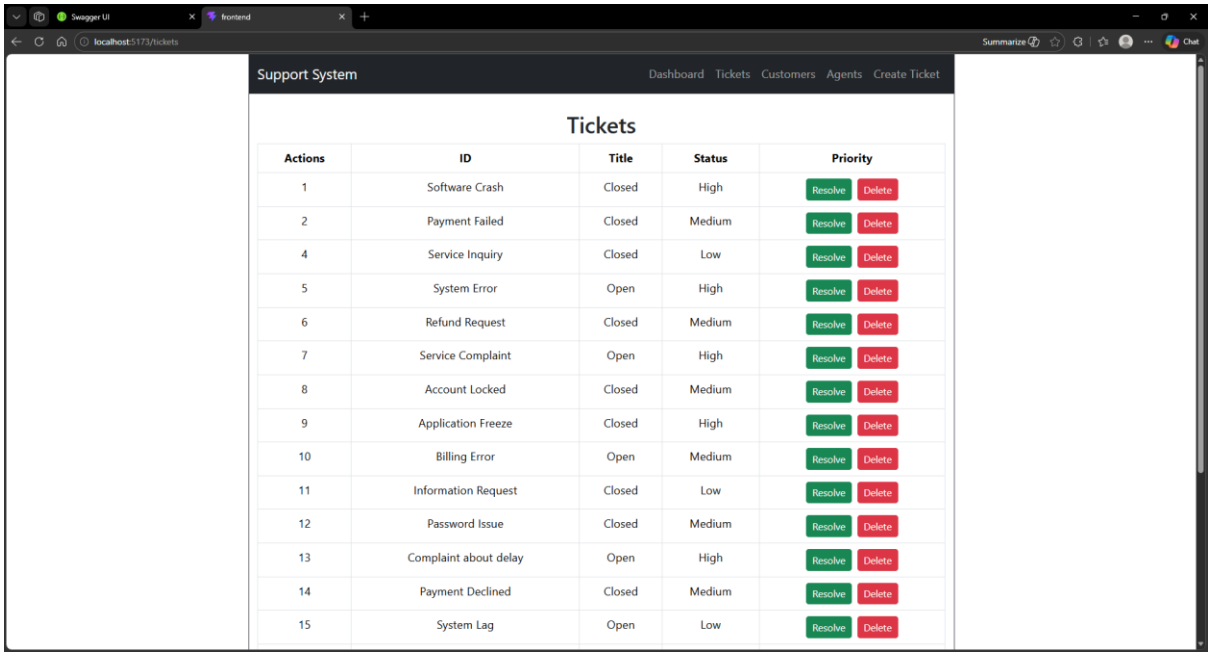
• Tickets Page
Displays all tickets in a tabular format, including ticket ID, title, status, and priority.

• Create Ticket Component
Provides a form for users to create new support tickets.

• Customers Page
Displays customer details such as name, email, and phone number.

• Agents Page
Displays agent information.

• API Service Module
Handles all HTTP requests to the backend using the Fetch API.



Tickets Page

4.3. Backend Component Breakdown:

The backend is implemented using ASP.NET Core Web API and follows a layered architecture.

The main components are:

• Controller Layer
Handles incoming HTTP requests and returns appropriate responses.

Example: TicketsController.

• Service Layer
Contains business logic and processes data before sending it to the repository.

• Repository Layer
Handles all database interactions using Entity Framework Core.

• Data Layer
Represents the database context and entity models such as Ticket, Customer, and Agent.

```
using Microsoft.AspNetCore.Mvc;
using CustomerSupportAPI.Services;
using CustomerSupportAPI.Models;

namespace CustomerSupportAPI.Controllers
{
    [Route("api/[controller]")]
    [ApiController]
    public class TicketsController : ControllerBase
    {
        private readonly TicketService _service;

        public TicketsController(TicketService service)
        {
            _service = service;
        }

        [HttpGet]
        public IActionResult GetAllTickets()
        {
            return Ok(_service.GetAllTickets());
        }

        [HttpGet("{id}")]
        public IActionResult GetTicketById(int id)
        {
            return Ok(_service.GetTicketById(id));
        }

        [HttpPost]
        public IActionResult AddTicket(Ticket ticket)
        {
            _service.CreateTicket(ticket);
            return Ok("Ticket created successfully");
        }

        // DELETE Ticket
        [HttpDelete("{id}")]
        public IActionResult DeleteTicket(int id)
        {
            _service.DeleteTicket(id);
            return Ok("Ticket deleted successfully");
        }

        // RESOLVE Ticket
        [HttpPut("resolve/{id}")]
        public IActionResult ResolveTicket(int id)
        {
            var ticket = _service.GetTicketById(id);

            if (ticket == null)
            {
                return NotFound();
            }

            ticket.Status = "Closed";
            ticket.ResolvedDate = DateTime.Now;

            _service.UpdateTicket(ticket);

            return Ok("Ticket resolved successfully");
        }
    }
}
```

TicketController.cs

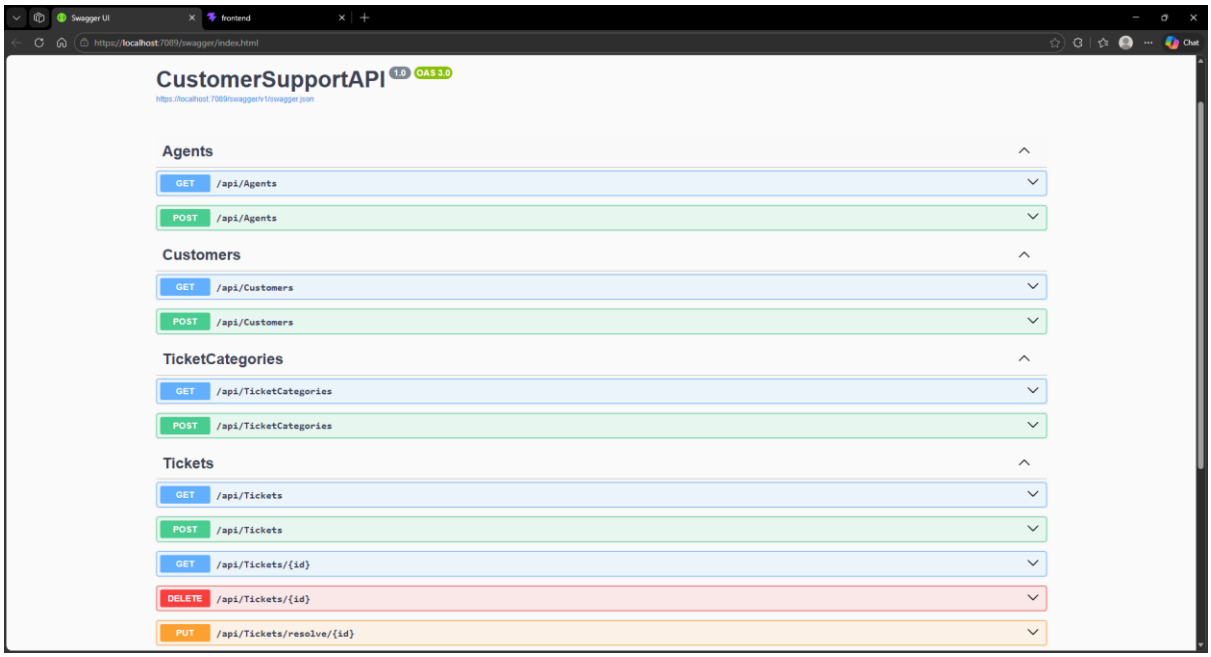
4.4. API Design:

The backend exposes RESTful APIs that allow the frontend to perform CRUD (Create, Read, Update, Delete) operations on tickets and other entities.

These APIs follow standard HTTP methods such as GET, POST, PUT, and DELETE.

4.5. API Endpoints Table

Method	Endpoint	Description
GET	/api/Tickets	Retrieve all tickets
GET	/api/Tickets/{id}	Retrieve ticket by ID
POST	/api/Tickets	Create a new ticket
PUT	/api/Tickets/resolve/{id}	Mark ticket as resolved
DELETE	/api/Tickets/{id}	Delete a ticket



4.6. Request & Response Example:

Example: Creating a Ticket

Request (POST /api/Tickets)


```
{
"customerID": 1,
"agentID": 2,
"categoryID": 1,
"title": "Login Issue",
"description": "Unable to login to account",
"status": "Open",
"priority": "High",
"createdDate": "2026-03-17T10:00:00"
}
```


Response: "Ticket created successfully"

4.7. Data Flow Between Components:

The frontend components communicate with the backend through API calls. When a user interacts with the UI, such as creating or resolving a ticket, the request is sent to the backend API.

The controller receives the request and forwards it to the service layer. The service layer processes the data and interacts with the repository layer, which performs database operations.

The response is then sent back to the frontend and displayed to the user

5. Database Design & Storage Optimization

Overview:

The database for the Customer Support Ticket Management System is designed using a relational model in Microsoft SQL Server. It stores all the core data related to customers, agents, tickets, and ticket categories.

The design ensures data consistency, reduces redundancy, and supports efficient querying for reporting and analytics.

Tables in the Database:

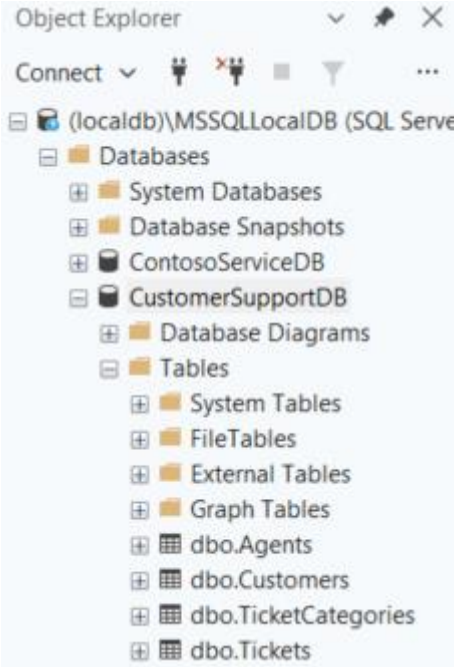
The system consists of the following main tables:

• Customers
Stores customer details such as CustomerID, FirstName, LastName, Email, Phone, and CreatedDate.

• Agents
Stores support agent details including AgentID, Name, Email, and role.

• Tickets
Stores all support tickets with fields such as TicketID, CustomerID, AgentID, CategoryID, Title, Description, Status, Priority, CreatedDate, and ResolvedDate.

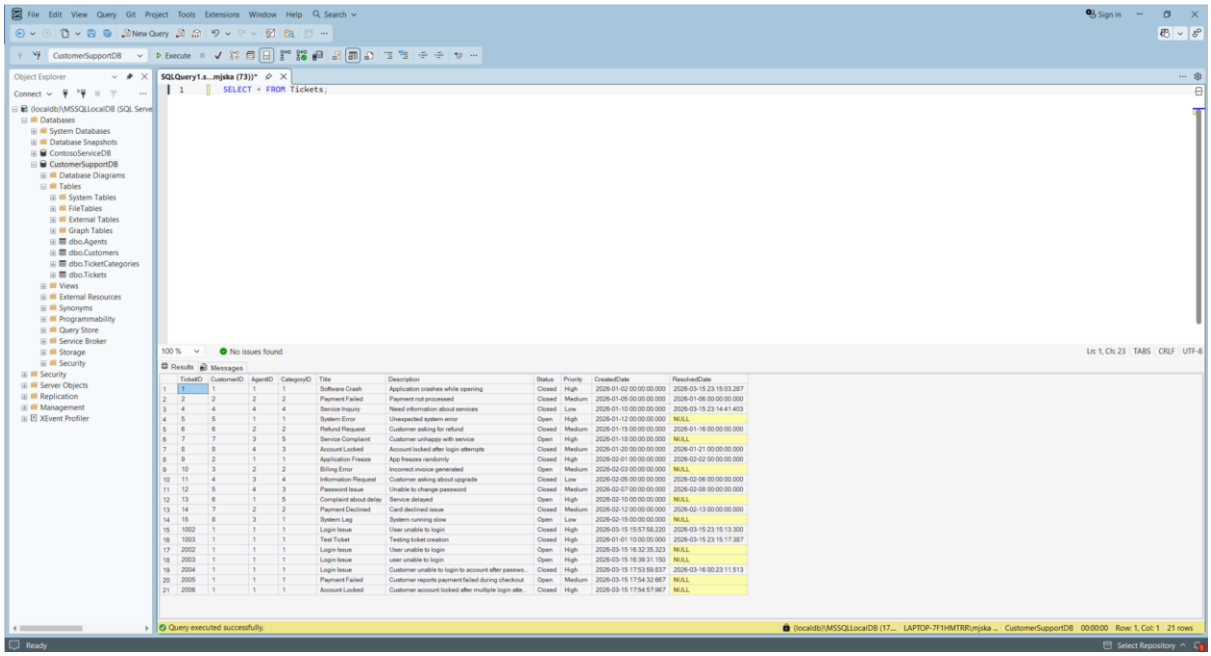
• TicketCategory
Stores different categories of tickets such as Technical, Billing, and General queries.



5.1. Table Structure Example (Tickets Table):

The Tickets table is the central table in the system and contains all ticket-related information.

Important fields include:
- TicketID (Primary Key)
- CustomerID (Foreign Key)
- AgentID (Foreign Key)
- CategoryID (Foreign Key)
- Title
- Description
- Status
- Priority
- CreatedDate
- ResolvedDate



Results | Messages

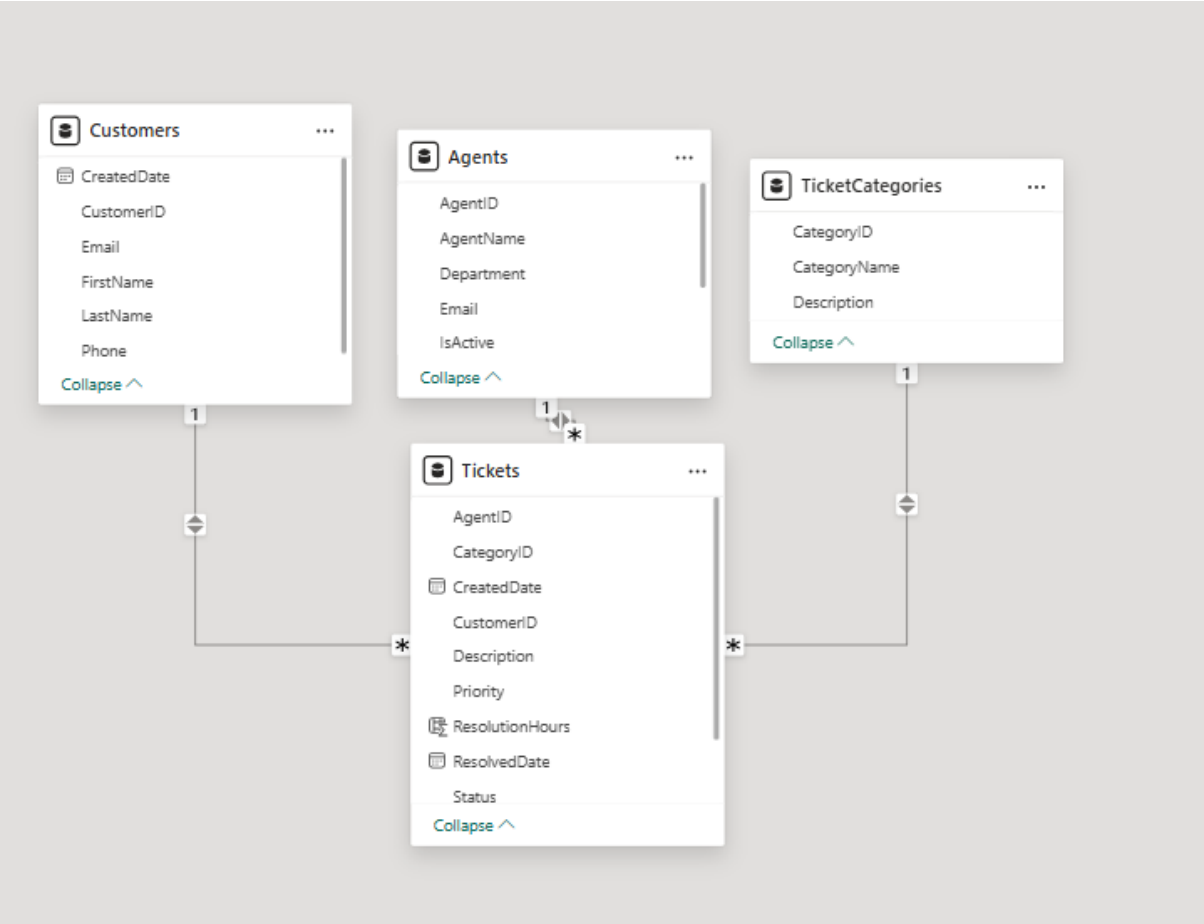
	TicketID	CustomerID	AgentID	CategoryID	Title	Description	Status	Priority	CreatedDate	ResolvedDate
1	1	1	1	1	Software Crash	Application crashes while opening	Closed	High	2026-01-02 00:00:00.000	2026-03-15 23:15:03.287
2	2	2	2	2	Payment Failed	Payment not processed	Closed	Medium	2026-01-05 00:00:00.000	2026-01-06 00:00:00.000
3	4	4	4	4	Service Inquiry	Need information about services	Closed	Low	2026-01-10 00:00:00.000	2026-03-15 23:14:41.403
4	5	5	1	1	System Error	Unexpected system error	Open	High	2026-01-12 00:00:00.000	NULL
5	6	6	2	2	Refund Request	Customer asking for refund	Closed	Medium	2026-01-15 00:00:00.000	2026-01-16 00:00:00.000
6	7	7	3	5	Service Complaint	Customer unhappy with service	Open	High	2026-01-18 00:00:00.000	NULL
7	8	8	4	3	Account Locked	Account locked after login attempts	Closed	Medium	2026-01-20 00:00:00.000	2026-01-21 00:00:00.000
8	9	2	1	1	Application Freeze	App freezes randomly	Closed	High	2026-02-01 00:00:00.000	2026-02-02 00:00:00.000
9	10	3	2	2	Billing Error	Incorrect invoice generated	Open	Medium	2026-02-03 00:00:00.000	NULL
10	11	4	3	4	Information Request	Customer asking about upgrade	Closed	Low	2026-02-05 00:00:00.000	2026-02-06 00:00:00.000
11	12	5	4	3	Password Issue	Unable to change password	Closed	Medium	2026-02-07 00:00:00.000	2026-02-08 00:00:00.000
12	13	6	1	5	Complaint about delay	Service delayed	Open	High	2026-02-10 00:00:00.000	NULL
13	14	7	2	2	Payment Declined	Card declined issue	Closed	Medium	2026-02-12 00:00:00.000	2026-02-13 00:00:00.000
14	15	8	3	1	System Lag	System running slow	Open	Low	2026-02-15 00:00:00.000	NULL
15	1002	1	1	1	Login Issue	User unable to login	Closed	High	2026-03-15 15:57:58.220	2026-03-15 23:15:13.300
16	1003	1	1	1	Test Ticket	Testing ticket creation	Closed	High	2026-01-01 10:00:00.000	2026-03-15 23:15:17.387
17	2002	1	1	1	Login Issue	User unable to login	Open	High	2026-03-15 16:32:35.323	NULL
18	2003	1	1	1	Login Issue	user unable to login	Open	High	2026-03-15 16:39:31.150	NULL
19	2004	1	1	1	Login Issue	Customer unable to login to account after passwo...	Closed	High	2026-03-15 17:53:59.837	2026-03-16 00:23:11.513
20	2005	1	1	1	Payment Failed	Customer reports payment failed during checkout	Open	Medium	2026-03-15 17:54:32.667	NULL
21	2006	1	1	1	Account Locked	Customer account locked after multiple login atte...	Closed	High	2026-03-15 17:54:57.967	NULL

5.2. Relationships between Tables:

The database follows a relational structure with the following relationships:

• One Customer can have many Tickets (One-to-Many)
• One Agent can handle many Tickets (One-to-Many)
• One Category can have many Tickets (One-to-Many)

Foreign keys are used to establish relationships between tables and maintain referential integrity.



The Entity Relationship Diagram (ERD) illustrates the relationships between the main entities in the system. The Tickets table acts as the central entity and connects with Customers, Agents, and TicketCategory through foreign keys.

This structure ensures efficient data retrieval and supports reporting and analytics features.

5.3. Sample SQL Queries:

The following SQL queries are used to retrieve and analyse ticket data:

1. Retrieve all tickets:
   SELECT * FROM Tickets
2. Retrieve open tickets:
   SELECT * FROM Tickets WHERE Status = 'Open'
3. Count tickets by priority:
   SELECT Priority, COUNT(*) FROM Tickets GROUP BY Priority
4. Calculate average resolution time:
   SELECT AVG(DATEDIFF(DAY, CreatedDate, ResolvedDate)) AS AvgResolutionDays FROM Tickets WHERE ResolvedDate IS NOT NULL

5.4. Storage Optimization Techniques:

The database design incorporates several optimization techniques:

• Normalization
Data is organized into multiple related tables to eliminate redundancy.

• Indexing
Primary keys are indexed to improve query performance.

• Efficient Data Types
Appropriate data types such as INT and DATETIME are used to optimize storage.

• Nullable Fields
ResolvedDate is nullable to allow tickets that are not yet resolved.

Integration with Backend:

The database is connected to the backend using Entity Framework Core. The DbContext class manages database connections and maps database tables to C# models.

CRUD operations are performed through the repository layer, ensuring clean separation between business logic and data access.

6. Reporting & Analytics

Power BI Dashboard:

Overview:

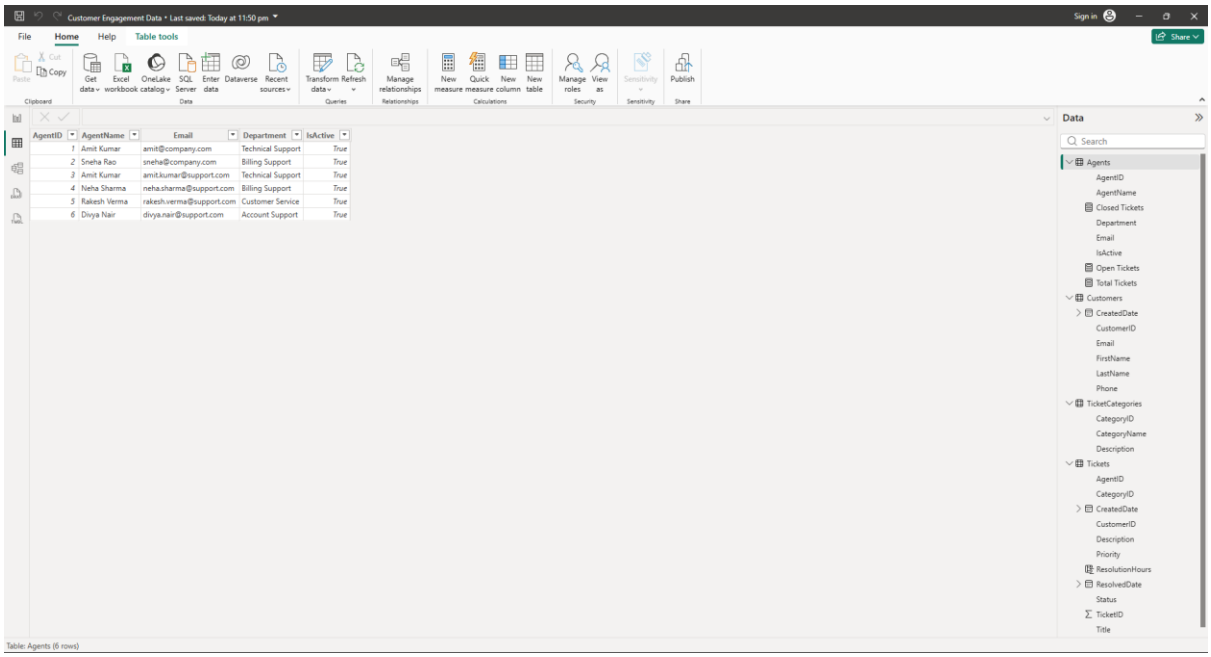
The database is connected to the backend using Entity Framework Core. The DbContext class manages database connections and maps database tables to C# models.

CRUD operations are performed through the repository layer, ensuring clean separation between business logic and data access.

Data Source Connection:

The Power BI dashboard is connected to the Microsoft SQL Server database. Data is imported from the Tickets table and related tables such as Customers and Agents.

The connection is established using the SQL Server connector in Power BI Desktop by providing the server's name and database name.



Dashboard Visualizations:

1. Tickets by Status:

A pie chart is used to display the distribution of tickets based on their status (Open, In Progress, Resolved). This helps in understanding the current workload and pending issues.

2. Tickets by Priority:

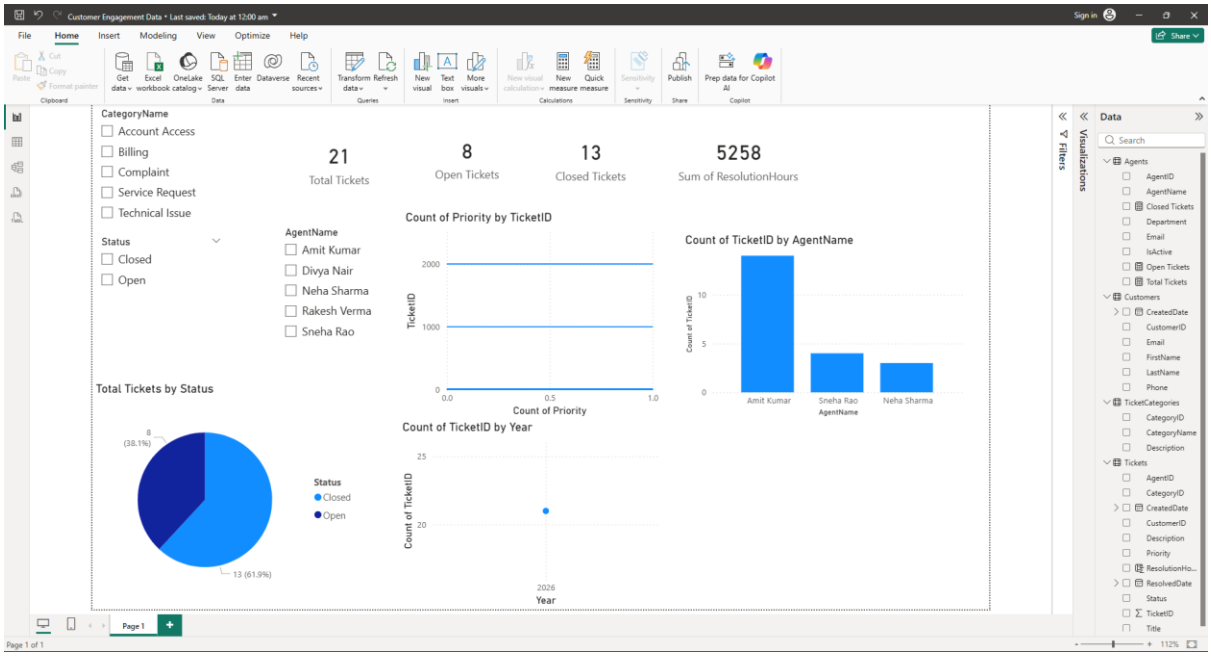
A pie chart is used to display the distribution of tickets based on their status (Open, In Progress, Resolved). This helps in understanding the current workload and pending issues.

3. Tickets Created Over Time:

A line chart displays the number of tickets created over time, helping to identify trends and peak support periods.

4. KPI Cards:

Multiple KPI cards are used to display the total number of tickets in the system, total number of open tickets in the system, total number of closed tickets in the system.



Power BI Dashboard

Integration with the System:

Power BI is integrated with the backend database, allowing seamless data flow from the application to the dashboard. This integration ensures that all ticket-related updates are reflected in the dashboard after refresh.

7.Conculsion:

This project successfully demonstrates the design and implementation of a Customer Engagement and Service Analytics System using SQL Server and Power BI. A normalized database schema was developed to ensure data integrity, consistency, and efficient data management across customers, tickets, categories, and agents.

The integration of Power BI enabled the creation of interactive dashboards that provide meaningful insights into ticket trends, agent performance, and resolution efficiency. These analytics support better decision-making and help organizations improve their customer service processes.

Overall, the project highlights the importance of combining well-structured databases with data visualization tools to build scalable and data-driven solutions for real-world business problems.